Ringo

Der private Raum eurer Hochzeit. Gäste werden eingeladen und sagen zu. Am Tag selbst landen alle Fotos per QR-Code in einem gemeinsamen Album und erscheinen live auf der Leinwand. Danach bleiben alle Erinnerungen an einem Ort – ohne App, ohne Konto für Gäste, ohne Social Media.

Das Projekt ist die Neuauflage des Hochzeits-App-Konzepts von 2014 (Ordner `../design/`, Android-Prototyp unter `../app/`). Es wurde konsequent auf die heutige Rolle von Smartphone und Social Media ausgerichtet.

Für wen	Was
Brautpaar	Verwaltung unter `/verwaltung`: Termine, Gäste & Einladungen, Rückmeldungen, Ablauf & Live-Steuerung, Nachrichten, Fragen/FAQ, Fotos, Gästebuch, Geschenke, Sitzplan, Drucksachen
Gäste	Persönlicher Link → Web-App: Zu-/Absage, Infos, Live-Ablauf, Fotos, Video-/Sprach-Gästebuch, Geschenke mit GiroCode, Sitzplatz, „Finde meine Fotos“, Fragen stellen, Geschichten für Reden
Trauzeugen / Organisatoren	Zusätzlich ein geheimer Team-Bereich: Überraschungen, Programm-Freigaben, Live-Steuerung, gesammelte Geschichten, Rede-Helfer, Absprachen
Alle am Tag selbst	QR-Code auf dem Tisch → `/foto/…` → knipsen, fertig. Live-Fotowand auf Beamer/TV unter `/wand/…`
Design	Eigenes Logo statt Namensschriftzug möglich. Das Brautpaar wählt eines von 8 Designs für alle Gästeseiten (Klassisch, Hochzeitszeitung, Sofortbild, Konfetti-Pop, Standesamt, Ballnacht, Wunderkerzen, Blütenmeer)
Betreiber	Betreiber-Konsole `/betreiber`: alle Hochzeiten, Speicherverbrauch, Paket freischalten

Schnellstart (lokal)

Voraussetzung: **Node.js 22.13 oder neuer.** Die Datenbank ist das eingebaute `node:sqlite`. Native Abhängigkeiten gibt es nicht.

```
npm install
npm run demo        # legt die Demo-Hochzeit „Lena & Jonas“ an und gibt alle Links aus
npm start           # http://localhost:3000
```

Demo-Login für das Brautpaar: `demo@ringo.test` / `demo1234`. `npm run demo` gibt außerdem die Gäste-Links (z. B. Trauzeugin Anna mit Team-Bereich), den Foto-Upload-Link und den Link zur Fotowand aus.

Dokumentation

Dokument	Inhalt
docs/Handbuch-Brautpaar.md	Schritt für Schritt von der Registrierung bis zu den Dankeskarten
docs/Anleitung-Gaeste.md	Kurzanleitung für Gäste und Trauzeugen (zum Weitergeben)
docs/Betriebshandbuch.md	Installation, Konfiguration, HTTPS, Backup, Updates, Datenschutz, Fehlersuche
docs/Architektur.md	Technischer Aufbau, Datenmodell, Sicherheitsmodell, API, Tests
docs/Broschuere.pdf	Broschüre für Kundinnen und Kunden (Quelle: `docs/Broschuere.html`)

Logo: `public/img/ringo.svg` (Wortmarke), `public/img/ringo-hell.svg` (für dunkle Flächen), `public/img/logo.svg` (App-Symbol). Neu erzeugen mit `node scripts/make-logo.mjs`; dabei wird der Schriftzug in Vektorpfade umgewandelt, damit das Logo überall ohne Schriftart gleich aussieht.

Broschüre neu bauen: `python docs/build-brochure.py` (Vorlage: `docs/broschuere/vorlage.html`).

Alle Handbücher liegen zusätzlich als PDF in `docs/` (neu erzeugen: `python docs/md2pdf.py`, benötigt `pip install markdown pymdown-extensions` und Chrome).

Tests

```
npm test              # 26 API-Tests (isolierte Temp-Datenbank)
npm run demo && npm start   # dann in zweitem Terminal:
npm run test:e2e      # alle Seiten in Chrome als Brautpaar/Gast/Team/QR/Wand, inkl. aller PDF-Exporte
npm run test:browser  # Bildverarbeitung + Upload, Gesichtserkennung, Video-/Sprachaufnahme
```

Projektstruktur

```
server/           Express-Server, SQLite, API (routes/), PDFs, Sitzplan, KI, Backup, Demo-Daten
public/           Frontend ohne Build-Schritt (ES-Module): Landingpage, Verwaltung, Gäste-App, QR-Upload, Fotowand
docs/             Handbücher, Broschüre, Screenshots
tests/            API-Tests (node:test) und Browser-Tests (puppeteer-core + lokales Chrome)
Dockerfile, docker-compose.yml, Caddyfile, .env.example   Betrieb
```